

Project Report: Multiple Element Identification

Batch DOM Collection Traversal, Attribute Extraction, and State Manipulation

Target OS: Arch Linux x86_64 • Engine: Mozilla Firefox & GeckoDriver • Date: September 2026

1 Executive Summary

Automated end-to-end testing frequently demands interrogating groups of identical or functionally related DOM elements rather than isolated single nodes. This project report details the design, implementation, and empirical validation of an automated batch-processing test suite deployed on **Arch Linux** utilizing **Selenium 4.x**, driving **Mozilla Firefox** through the native **GeckoDriver** interface.

The test implementation exercises the core distinction between singleton discovery (`find_element`) and collection retrieval (`find_elements`). The suite comprises two distinct phases:

- Bulk Anchor Interrogation & Tabulation:** Querying the complete set of anchor tags (`<a>`) on an industry-standard test fixture (<https://the-internet.herokuapp.com>), parsing inner text payloads, extracting destination hyperlinks (`href`), cataloging empty/graphical nodes, and applying programmatic substring filtering.
- Multi-Element Interactive State Verification:** Locating an input collection of checkboxes, evaluating initial boolean selection states via W3C property endpoints (`is_selected()`), executing programmatic toggle events, and formally asserting that state inversion occurred across every item in the collection.

2 System Architecture & Environment Topology

The test automation framework was deployed directly on a POSIX-compliant Arch Linux environment, utilizing standard system package binaries for stability and process lifecycle determinism.

Component Layer	Deployment Specification	Operational Role
Host OS	Arch Linux (x86_64 kernel rolling)	Host execution runtime and process supervisor
Web Browser	Mozilla Firefox (pacman -S firefox)	Gecko layout rendering engine
WebDriver Binary	GeckoDriver (/usr/bin/geckodriver)	Native W3C WebDriver wire protocol server
Scripting Layer	Python 3.x (venv isolated)	High-level test logic and list orchestrator
Automation API	Selenium 4.x WebDriver API	DOM traversal, collection queries, event dispatch
Target SUT	The Internet (Herokuapp Testbed)	Canonical web fixture for automation verification

Table 1: Test runtime environment and operational layer mapping.

3 Technical Implementation & Collection Methodology

3.1 Architectural Distinction: `find_element` vs. `find_elements`

The W3C WebDriver specification defines two distinct element discovery paradigms:

- **`find_element()`**: Queries the DOM for a locator match. If found, it returns the *first matching instance* as a single `WebElement`. If no matching node exists, it halts execution and raises a `NoSuchElementException`.
- **`find_elements()`**: Traverses the DOM subtree and returns a Python `list` containing *all* matching `WebElement` instances. If no elements match, it returns an empty list `[]` without raising an exception, making it the foundational primitive for bulk data extraction, existence polling, and dynamic component verification.

3.2 Phase 1: Anchor Node Parsing & Attribute Extraction

The first phase interrogates the root index page to discover all interactive links:

1. **Collection Gathering**: The locator `By.TAG_NAME, "a"` traverses the document tree, capturing 46 total anchor references.
2. **DOM Property Sanitization**: The loop retrieves `.text.strip()` and `.get_attribute("href")`. Nodes with empty text (such as absolute-positioned GitHub fork ribbons or decorative icon wrappers) are cleanly categorized without causing runtime faults.
3. **Stale Element Resilience**: Iterating across dynamic DOM collections exposes scripts to DOM refreshes. A targeted `try...except StaleElementReferenceException` block ensures resilient inspection.
4. **Functional Filtering**: Using list comprehensions, the script extracts links whose display names begin with specific prefixes (e.g., character "C"), validating that structured sub-queries can be derived from raw web collections.

3.3 Phase 2: Checkbox Manipulation & Dynamic Assertion

The second phase targets the `/checkboxes` route to test multi-element manipulation:

1. **Scoped Descendant Targeting**: The selector `#checkboxes input[type='checkbox']` isolates interactive inputs inside the dedicated container, preventing accidental interaction with navigation or footer links.
2. **State Querying**: The initial checked status (S_0) of each element is sampled using `box.is_selected()` identifying that Checkbox #1 is initially `False` (unchecked) while Checkbox #2 is `True` (checked).
3. **Interactive Dispatch & Inversion**: The script triggers a click event on each element, captures the new state (S_1), and verifies through a programmatic assertion that:

$$S_1 = \neg S_0 \quad \wedge \quad S_1 \neq S_0 \tag{1}$$

This confirms bidirectional responsiveness without false-positive assumptions.

4 Verification Matrix & Collection Statistics

5 Runtime Execution Log & Telemetry Output

Target Collection	Locator Strategy	Total Matched	Valid / Active	Extracted Metric
All Page Links	By.TAG_NAME, "a"	46	45 Text / 1 Graphic	3 Prefix-Filtered ('C')
Interactive Checkboxes	By.CSS_SELECTOR	2	2 Toggled	100% State Inversion (S_0 6=

Table 2: Quantitative results of multiple element extraction and assertion.

Target Input	Initial State (S_0)	Post-Click (S_1)	Assertion Verification
Checkbox #1	False (Unchecked)	True (Checked)	assert False != True → Passed
Checkbox #2	True (Checked)	False (Unchecked)	assert True != False → Passed

Table 3: Boolean state-transition matrix for the interactive checkbox collection.

The test automation harness executed cleanly within an active Python virtual environment on Arch Linux. Below is the direct terminal stream demonstrating link enumeration, formatted data extraction, sub-filtering, and checkbox state validation.

```
.venv > python Assignment_2.py
=====
STARTING ASSIGNMENT 2: MULTIPLE ELEMENT IDENTIFICATION
=====

[*] Navigating to: https://the-internet.herokuapp.com
[\checkmark] Landing page loaded successfully.

-----
PART 1: EXTRACTING AND PROCESSING ALL LINKS (TAG_NAME: 'a')
-----

[*] Total anchor ('<a>') elements detected on page: 46

No.   | Link Text                                     | Destination (href)
-----|-----|-----
2     | A/B Testing                                | https://the-internet.herokuapp.com/...
3     | Add/Remove Elements                       | https://the-internet.herokuapp.com/...
4     | Basic Auth                               | https://the-internet.herokuapp.com/...
5     | Broken Images                             | https://the-internet.herokuapp.com/...
6     | Challenging DOM                           | https://the-internet.herokuapp.com/...
7     | Checkboxes                                | https://the-internet.herokuapp.com/...
8     | Context Menu                              | https://the-internet.herokuapp.com/...
9     | Digest Authentication                     | https://the-internet.herokuapp.com/...
10    | Disappearing Elements                     | https://the-internet.herokuapp.com/...
11    | Drag and Drop                             | https://the-internet.herokuapp.com/...
12    | Dropdown                                  | https://the-internet.herokuapp.com/...
13    | Dynamic Content                           | https://the-internet.herokuapp.com/...
14    | Dynamic Controls                          | https://the-internet.herokuapp.com/...
15    | Dynamic Loading                           | https://the-internet.herokuapp.com/...
16    | Entry Ad                                  | https://the-internet.herokuapp.com/...
17    | Exit Intent                               | https://the-internet.herokuapp.com/...
18    | File Download                             | https://the-internet.herokuapp.com/...
19    | File Upload                               | https://the-internet.herokuapp.com/...
20    | Floating Menu                             | https://the-internet.herokuapp.com/...
21    | Forgot Password                           | https://the-internet.herokuapp.com/...
22    | Form Authentication                       | https://the-internet.herokuapp.com/...
23    | Frames                                    | https://the-internet.herokuapp.com/...
24    | Geolocation                               | https://the-internet.herokuapp.com/...
25    | Horizontal Slider                         | https://the-internet.herokuapp.com/...
26    | Hovers                                    | https://the-internet.herokuapp.com/...
27    | Infinite Scroll                           | https://the-internet.herokuapp.com/...
28    | Inputs                                    | https://the-internet.herokuapp.com/...
29    | JQuery UI Menus                           | https://the-internet.herokuapp.com/...
30    | JavaScript Alerts                         | https://the-internet.herokuapp.com/...
31    | JavaScript onload event error             | https://the-internet.herokuapp.com/...
32    | Key Presses                               | https://the-internet.herokuapp.com/...
33    | Large & Deep DOM                           | https://the-internet.herokuapp.com/...
34    | Multiple Windows                          | https://the-internet.herokuapp.com/...
```

```

35 | Nested Frames | https://the-internet.herokuapp.com/...
36 | Notification Messages | https://the-internet.herokuapp.com/...
37 | Redirect Link | https://the-internet.herokuapp.com/...
38 | Secure File Download | https://the-internet.herokuapp.com/...
39 | Shadow DOM | https://the-internet.herokuapp.com/...
40 | Shifting Content | https://the-internet.herokuapp.com/...
41 | Slow Resources | https://the-internet.herokuapp.com/...
42 | Sortable Data Tables | https://the-internet.herokuapp.com/...
43 | Status Codes | https://the-internet.herokuapp.com/...
44 | Typos | https://the-internet.herokuapp.com/...
45 | WYSIWYG Editor | https://the-internet.herokuapp.com/...
46 | Elemental Selenium | http://elementalselenium.com/

```

```
-----
[\checkmark] Summary of Link Extraction:
```

```

- Total Links Identified : 46
- Links with Visible Text : 45
- Links without Text/Icons : 1

```

```
[*] Filtering Links: Finding all links starting with the letter 'C'...
```

```

-> Found 3 matching links:
- Challenging DOM
- Checkboxes
- Context Menu

```

```
-----
PART 2: WORKING WITH A COLLECTION OF INTERACTIVE ELEMENTS (CHECKBOXES)
-----
```

```
[*] Navigating to: https://the-internet.herokuapp.com/checkboxes
```

```
[\checkmark] Found 2 checkbox elements in the collection.
```

```
[Checkbox #1] Initial checked status: False
```

```
    Toggling Checkbox #1 via .click()...
```

```
    New checked status : True
```

```
    [\checkmark] State change verified successfully.
```

```
[Checkbox #2] Initial checked status: True
```

```
    Toggling Checkbox #2 via .click()...
```

```
    New checked status : False
```

```
    [\checkmark] State change verified successfully.
```

```
[SUCCESS] Assignment 2 completed: successfully gathered and operated on element lists!
```

```
[*] Closing browser session...
```

```
[\checkmark] WebDriver terminated cleanly.
```

6 Engineering Takeaways & Best Practices

The successful execution of Assignment 2 highlights several critical architectural patterns for multi-element test automation:

- **Safe Existence Inquiries:** Using `find_elements()` is the recommended pattern to test whether an element exists without incurring exception overhead or altering the implicit wait timeline. An empty list simply indicates absence (`len = 0`).
- **Guaranteed Iteration Safety:** In dynamic, asynchronous single-page web applications (SPAs), iterating over elements returned from a single collection query can trigger `StaleElementReferenceException` if the page re-renders mid-loop. Defensive handling or re-querying by index prevents brittle test suites.
- **Explicit Scope Chaining:** When querying multiple elements, narrowing the root container using parent CSS selectors (such as `#checkboxes input`) reduces DOM traversal depth and eliminates false positives caused by similarly tagged components elsewhere in the layout.
- **Process Hygiene in POSIX Environments:** Wrapping the browser lifecycle in a `try...finally` construct ensures that the `GeckoDriver` daemon terminates cleanly and releases Linux file descriptors and virtual display sockets upon suite completion.